Week 8 Worked Companion

Using the worked companion

Attempt the student exercises independently first. This companion contains hints and complete solutions. Read only as far as needed, then return to your attempt.

Examples are original teaching examples. Follow each official link for the source problem and its complete contract.

Student exercises

Instructor guide

Download Week 8 code

55 · Staged hints

LeetCode 55: Jump Game · Medium

H1. What is the farthest reachable index so far?

H2. Can later values repair an unreachable gap?

H3. Update farthest only while index is reachable.

55 · Correctness and cost

Reachable positions form the interval 0..farthest. Allowed jumps include every shorter distance, preserving an interval; an index beyond it cannot be crossed.

n is length. O(n) time,O(1) space.

55 · Counterexamples and checks

In [2,3,0,0,0], taking the longest first jump stops at the zero at index 2. Exploring reachable index 1 also exposes a path to the last index. The algorithm maintains the range reachable from all eligible positions.

```
nums=[2,0,1,0] → True
```

Predict the state and return value before running this example. Then check the boundary cases above.

55 · Implementation

```
def can_jump(nums):
    farthest = 0
    for index, step in enumerate(nums):
        if index > farthest:
            return False
        farthest = max(farthest, index + step)
    return True
```

45 · Staged hints

LeetCode 45: Jump Game II · Medium

H1. Where does the current jump frontier end?

H2. Which positions must be considered before committing the next jump?

H3. end is the current frontier, farthest the next; increment jumps only at index==end.

45 · Correctness and cost

Scan every reachable position in the current level before fixing the next frontier. These are BFS levels in an implicit unweighted graph, so the jump count is minimal.

n is length. O(n) time,O(1) auxiliary space.

45 · Counterexamples and checks

The target is n-1. Processing the last index overcounts; [0] gives zero. Unreachable inputs are outside this contract.

`nums=[2,3,0,1,0] → 2`

Predict the state and return value before running this example. Then check the boundary cases above.

45 · Implementation

```
def min_jumps(nums):
    end = farthest = jumps = 0
    for index in range(len(nums) - 1):
        farthest = max(farthest, index + nums[index])
        if index == end:
            jumps += 1
            end = farthest
    return jumps
```

134 · Staged hints

LeetCode 134: Gas Station · Medium

H1. Can choosing a start repair a total fuel deficit?

H2. After first failing from s through i, can an intermediate start work?

H3. Accumulate total difference and candidate tank; restart at i+1 after tank becomes negative.

134 · Correctness and cost

At the first failure over s..i, each earlier prefix s..j-1 is nonnegative. Removing such a prefix leaves j..i negative too, eliminating all intermediate starts. If total is nonnegative, the final candidate also has enough surplus to wrap through the eliminated prefixes.

n is station count. O(n) time,O(1) space.

134 · Counterexamples and checks

Choosing the largest gas supply is unjustified: gas=[5,0,4],cost=[6,1,2] fails at station 0; station 2 is the unique valid start.

```
gas=[1,4,2], cost=[3,1,3] → 1
```

Predict the state and return value before running this example. Then check the boundary cases above.

134 · Implementation

```
def gas_start(gas, cost):
    total = tank = start = 0
    for index, (supply, use) in enumerate(zip(gas, cost)):
        total += supply - use
        tank += supply - use
        if tank < 0:
            start, tank = index + 1, 0
    return start if total >= 0 else -1
```

200 · Staged hints

LeetCode 200: Number of Islands · Medium

Repeat

H1. What happens if visited land starts another component?

H2. Are diagonal neighbors connected?

H3. Start a stack traversal at each unseen land cell and mark its whole component.

200 · Correctness and cost

A traversal reaches exactly its starting component. Marked components cannot restart; the number of starts equals the component count.

R,C are dimensions. O(RC) time and space; general adjacency lists take O(V+E) time.

200 · Counterexamples and checks

Use == for values; is tests identity. Diagonal land in a 2x2 grid forms two components.

```
grid=[["1","0","1"],["1","0","0"]] → 2
```

Predict the state and return value before running this example. Then check the boundary cases above.

200 · Implementation

```
def islands(grid):
    rows, cols = len(grid), len(grid[0])
    seen, count = set(), 0
    for row in range(rows):
        for col in range(cols):
            if grid[row][col] != "1" or (row, col) in seen:
                continue
            count += 1
            stack = [(row, col)]
            seen.add((row, col))
            while stack:
                r, c = stack.pop()
                for nr, nc in ((r-1,c),(r+1,c),(r,c-1),(r,c+1)):
                    if (0 <= nr < rows and 0 <= nc < cols
                            and grid[nr][nc] == "1" and (nr,nc) not in seen):
                        seen.add((nr,nc))
                        stack.append((nr,nc))
    return count
```

Execution and sources

```
cd code
python demo_15.py
python demo_16.py
python verify.py
```

Standard-library imports are in `solutions.py`. On LeetCode, adapt the function to the official method signature.

Erik Demaine and Srini Devadas, Introduction to Algorithms, MIT 6.006 (Fall 2011).

Checkpoint answers and reasoning

1. No. The reachable boundary remains 2.
2. 2, following 0→1→4.
3. Zero-based index 1. The tank after each leg is 3,2,0.
4. `0`. The start is already the last position, so no jump is counted.

Concept-check explanations

A successful example does not prove a greedy rule. Every position in the current reachable range must be considered before fixing the next level’s boundary.

Record performance on a familiar problem separately from performance under changed conditions. In the mixed assessment, select a method using the contract, state, and correctness argument.

Why the final start completes the circuit

Let $d[i]=gas[i]-cost[i]$, with prefix sums $P[0]=0$ and $P[k]=d[0]+\ldots+d[k-1]$. Resetting the candidate when tank becomes negative records a new smaller prefix sum. The final start s is a position of a minimum prefix sum.

After s, accumulated fuel is $P[k]-P[s]$, which is nonnegative. After wrapping to an earlier k, it is $P[n]-P[s]+P[k]$. If $P[n]\geq 0$, this is also nonnegative because $P[k]\geq P[s]$. Thus a nonnegative total guarantees that the final candidate completes the circuit.